

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Facultad de Ciencias Puras

Ingeniería Informática

INF761 — DESARROLLO DE VIDEOJUEGOS

GUÍA DE LABORATORIO N° 10

IA de Enemigos en **Pac-Man**

Blinky · Pinky · Inky · Clyde

Algoritmos canónicos del arcade clásico (1980)

Tipo de proyecto: 2D arcade (Pac-Man clásico)

Motor: Unity 6.x (o Unity 2022 LTS en adelante)

Tecnologías: Tilemap · 2D Physics · IA basada en grilla

Duración estimada: 4-6 horas (2 sesiones teórico-prácticas)

Docente: M.Sc. Huáscar Fedor Gonzales Guzmán

1. Objetivos de la Práctica

1.1 Objetivo general

Implementar una réplica fiel del videojuego Pac-Man (1980) en Unity, reproduciendo los algoritmos canónicos de movimiento e Inteligencia Artificial de los cuatro fantasmas originales (Blinky, Pinky, Inky y Clyde), incluyendo sus personalidades distintivas, los modos de comportamiento global (Scatter/Chase/Frightened) y las mecánicas clásicas del laberinto (ghost house, túneles laterales, power pellets).

1.2 Objetivos específicos

- Diseñar un sistema de movimiento basado en grilla que respete la lógica de "intersecciones" del Pac-Man original.
- Implementar el control del Pac-Man con buffer de input direccional y detección de paredes mediante raycasting.
- Programar los algoritmos de IA específicos de cada fantasma siguiendo los patrones documentados del arcade original.
- Implementar la máquina de estados global Scatter/Chase con sus tiempos canónicos (7s, 20s, 7s, 20s, 5s, 20s, 5s, ∞).
- Implementar el estado Frightened activado por los power pellets, con cambio visual y reversión de dirección.
- Programar la ghost house (casa de fantasmas) con orden de salida controlado por contador de pellets.
- Implementar los túneles laterales con wrap-around y reducción de velocidad.
- Aplicar el patrón "target tile" como mecanismo unificado para todos los comportamientos de fantasma.

Competencia desarrollada

Al finalizar esta guía, el estudiante será capaz de analizar e implementar IAs clásicas de videojuegos comprendiendo cómo personalidades emergentes complejas surgen de reglas simples por agente. Esta es la base conceptual de IAs modernas como squads en shooters tácticos o grupos de NPCs en juegos de mundo abierto.

2. Marco Teórico

2.1 Pac-Man: historia y relevancia técnica

Pac-Man fue lanzado por Namco en mayo de 1980 y se convirtió en un fenómeno cultural y un caso de estudio fundamental en el diseño de IA para videojuegos. Lo que hizo revolucionario al juego no fue su jugabilidad básica (comer puntos en un laberinto), sino la sensación de que cada fantasma tenía una "personalidad" propia.

Este efecto no es casualidad: cada uno de los cuatro fantasmas implementa un algoritmo de targeting distinto, diseñado conscientemente por Toru Iwatani y su equipo para que el jugador perciba una IA cooperativa cuando en realidad cada fantasma actúa de forma independiente. Esta es una lección clave de diseño: comportamiento emergente complejo a partir de reglas simples por agente.

2.2 Movimiento basado en grilla

Pac-Man y los fantasmas no se mueven con física continua: cada entidad ocupa una casilla (tile) y se desplaza de casilla en casilla. Las decisiones de IA solo se toman en las intersecciones (tiles donde hay más de dos direcciones posibles).

Las características técnicas clave de este sistema son:

- Cada tile mide típicamente 1 unidad de mundo en Unity (o 8 píxeles en el arcade original).
- El movimiento es continuo dentro del tile actual hacia el centro del siguiente, pero la dirección solo se reevalúa al alcanzar el centro de un tile.
- Los fantasmas no pueden invertir su dirección de marcha (excepto en transiciones Scatter↔Chase y al activarse Frightened).
- La elección de dirección en intersecciones se basa en minimizar la distancia euclidiana al "target tile" de cada fantasma.

2.3 El patrón "Target Tile"

Todos los fantasmas usan el mismo algoritmo de movimiento: en cada intersección eligen la dirección (excluyendo su dirección inversa) cuyo siguiente tile esté más cerca, en línea recta euclidiana, de un punto objetivo llamado target tile. Lo que diferencia a cada fantasma es cómo calcula su target tile.

Idea clave: una sola lógica de movimiento, cuatro personalidades

Toda la apariencia de "cooperación inteligente" entre los fantasmas surge de que cada uno calcula su target tile de forma diferente. El algoritmo de elección de dirección es idéntico para

los cuatro. Esta separación entre mecánica (cómo moverse) y estrategia (a dónde apuntar) es un patrón de diseño que sigue vigente en IAs modernas.

2.4 Estados globales: Scatter, Chase y Frightened

Los fantasmas alternan entre tres modos globales que afectan a todos simultáneamente:

Modo Scatter (dispersión)

Cada fantasma huye hacia su esquina asignada del laberinto, donde patrulla en círculo. Este modo da al jugador respiros y permite que los fantasmas se separen evitando que se acumulen sobre Pac-Man.

Modo Chase (persecución)

Cada fantasma calcula su target tile según su lógica personal. Es el modo principal y dura más que Scatter.

Modo Frightened (asustado)

Activado cuando Pac-Man come un power pellet. Los fantasmas se vuelven azules, lentos, y eligen direcciones aleatorias en cada intersección. En este estado pueden ser comidos por Pac-Man.

Temporización canónica del arcade

Los tiempos de Scatter/Chase en el primer nivel del arcade original son:

Fase	Modo	Duración
1	Scatter	7 segundos
2	Chase	20 segundos
3	Scatter	7 segundos
4	Chase	20 segundos
5	Scatter	5 segundos
6	Chase	20 segundos
7	Scatter	5 segundos
8	Chase	Permanente

Reversión de dirección en transiciones

Cada vez que ocurre una transición Scatter↔Chase, todos los fantasmas invierten instantáneamente su dirección de movimiento (excepto los que están dentro de la ghost house). Esto es una pista visual deliberada que avisa al jugador del cambio de modo.

2.5 La Ghost House (casa de fantasmas)

La casa de fantasmas, ubicada en el centro del laberinto, es donde los fantasmas comienzan el nivel y donde regresan después de ser comidos. El orden y momento de salida está controlado por dos contadores:

- Contador de pellets global: cada pellet comido por Pac-Man cuenta hacia un umbral que libera al siguiente fantasma.
- Umbrales canónicos: Blinky comienza fuera, Pinky sale inmediatamente, Inky sale tras 30 pellets, Clyde tras 60 pellets.

3. Los Cuatro Fantasmas y sus Personalidades

Esta sección documenta los algoritmos canónicos de cada fantasma. Memorizarlos y entenderlos es esencial antes de comenzar la implementación, ya que serán traducidos directamente a código C#.

Blinky (rojo)

Personalidad: El perseguidor agresivo y líder del grupo.

Esquina Scatter: Esquina superior derecha

Lógica Chase: Su target tile es directamente la posición actual de Pac-Man.

Blinky es el más simple y directo. Persigue a Pac-Man en línea recta hacia su tile actual. Adicionalmente, en el arcade original cuando quedan pocos pellets en el laberinto Blinky entra en modo "Cruise Elroy": aumenta su velocidad por encima de la de Pac-Man, lo que lo vuelve casi imposible de escapar. En esta guía implementaremos también este comportamiento.

Pinky (rosa)

Personalidad: El emboscador. Intenta interceptar a Pac-Man cortándole el paso.

Esquina Scatter: Esquina superior izquierda

Lógica Chase: Su target tile es 4 tiles por delante de Pac-Man, en la dirección que Pac-Man mira.

El bug famoso de Pinky

Cuando Pac-Man mira hacia ARRIBA, el target de Pinky se calcula 4 tiles arriba Y 4 tiles a la izquierda (no solo 4 arriba). Esto es un bug del arcade original causado por un error de overflow en el código ensamblador del Z80. Los desarrolladores nunca lo corrigieron y se considera parte canónica del comportamiento. En esta guía lo reproduciremos para mantener la fidelidad.

Inky (cian)

Personalidad: El impredecible. Combina las posiciones de Blinky y Pac-Man.

Esquina Scatter: Esquina inferior derecha

Lógica Chase: Toma el tile 2 delante de Pac-Man, traza un vector desde Blinky hasta ese punto, y duplica el vector. Ese es su target.

El algoritmo de Inky es el más complejo. Geométricamente: si llamamos A al punto 2 tiles delante de Pac-Man y B a la posición de Blinky, el target de Inky es $A + (A - B) = 2A - B$. Esto hace que Inky cambie radicalmente su comportamiento según dónde esté Blinky: si Blinky está lejos de Pac-Man, Inky se dispara hacia afuera; si Blinky está pegado a Pac-Man, Inky lo acompaña.

Clyde (naranja)

Personalidad: El tonto, o el cobarde. Cambia de estrategia según la distancia.

Esquina Scatter: Esquina inferior izquierda

Lógica Chase: Si está a MÁS de 8 tiles de Pac-Man, su target es Pac-Man. Si está a 8 tiles o menos, su target es su esquina de Scatter.

Clyde alterna entre perseguir y huir, generando un comportamiento errático que confunde al jugador. Su nombre alternativo en algunas versiones es "Pokey" precisamente por esa actitud impredecible.

4. Requisitos Previos

4.1 Software

- Unity Hub instalado.
- Unity Editor 2022.3 LTS o superior (recomendado: Unity 6.x).
- Visual Studio 2022 Community o Visual Studio Code con extensiones de C# y Unity.
- Git (opcional, recomendado para versionar el proyecto).

4.2 Conocimientos previos

- Sintaxis intermedia de C# (clases, enums, polimorfismo, abstract).
- Componentes básicos de Unity: GameObject, MonoBehaviour, Transform, Rigidbody2D, Collider2D.
- Familiaridad con la vista Scene, Hierarchy, Inspector y Project.
- Conceptos vistos en guías anteriores: input, animaciones, prefabs, scene management.

4.3 Plantilla del proyecto

Crearemos un proyecto desde cero usando la plantilla 2D Universal de Unity. No necesitaremos paquetes adicionales más allá de los incluidos por defecto, pero recomendamos verificar que el paquete 2D Tilemap Editor esté instalado (viene preinstalado en 2D Universal).

Sobre el uso de sprites y assets gráficos

Para mantener el foco en la IA, esta guía utilizará primitivas simples (cuadrados de colores) en lugar de sprites pixel-art del Pac-Man original. Al final ofrecemos como ejercicio adicional incorporar sprites animados. Los assets gráficos del arcade original están protegidos por copyright de Bandai Namco.

5. Configuración Inicial del Proyecto

Paso 5.1

Crear el proyecto

1. Abre Unity Hub y haz clic en New project.
2. Selecciona la plantilla Universal 2D.
3. Nómbralo Pacman.
4. Establece la ubicación y haz clic en Create project.

Paso 5.2

Organizar la estructura de carpetas

Dentro de Assets, crea la siguiente estructura:

```
Assets/
├── _Project/
│   ├── Scenes/
│   ├── Scripts/
│   │   ├── Core/           (Grid, Tiles, Constants)
│   │   ├── Player/        (PacmanController)
│   │   ├── Ghosts/        (clase base + 4 fantasmas)
│   │   ├── Managers/      (GameManager, ModeManager)
│   │   └── World/          (Pellet, PowerPellet, Tunnel)
│   ├── Prefabs/
│   ├── Materials/
│   └── Sprites/
```

Paso 5.3

Configurar la cámara y la escena

5. Crea una nueva escena en _Project/Scenes y nómbrala Pacman_Main.
6. Selecciona la Main Camera en la jerarquía.
7. Configura su Transform:

```
Position: (14, -15, -10)
Rotation: (0, 0, 0)
Scale:    (1, 1, 1)
```

8. En el componente Camera, configura:

- Projection: Orthographic
- Size: 15
- Background: color negro puro (#000000)

Sistema de coordenadas que usaremos

Trabajaremos con el laberinto en cuadrante positivo X negativo Y (es decir, las filas crecen hacia abajo). El laberinto del arcade mide 28 columnas × 31 filas. La esquina superior izquierda del laberinto será (0, 0). La cámara apunta al centro: (14, -15).

6. Construcción del Laberinto

Construiremos el laberinto basado en datos: un script generará tanto los muros visuales como la representación lógica del laberinto en memoria. Esto facilita modificar el diseño y mantener sincronizadas la representación visual y la lógica de colisión/pathfinding.

6.1 Convención de codificación del laberinto

Cada tile del laberinto se codifica con un carácter:

Carácter	Significado
#	Muro (no transitable)
.	Pellet (punto pequeño)
o	Power Pellet (punto grande)
	Tile vacío transitable (sin punto)
-	Puerta de la ghost house
G	Spawn de fantasmas (transitable, sin pellet)

Paso 6.2

Crear el script GridConstants

Primero definimos constantes globales que usaremos en todo el proyecto. Crea el script Scripts/Core/GridConstants.cs:

```
using UnityEngine;

public static class GridConstants
{
    public const int COLS = 28;
    public const int ROWS = 31;
    public const float TILE_SIZE = 1f;

    // Direccion de los 4 movimientos posibles
    public static readonly Vector2Int UP = new Vector2Int(0, -1);
    public static readonly Vector2Int DOWN = new Vector2Int(0, 1);
    public static readonly Vector2Int LEFT = new Vector2Int(-1, 0);
    public static readonly Vector2Int RIGHT = new Vector2Int(1, 0);

    public static readonly Vector2Int[] DIRS_CW = { UP, RIGHT, DOWN, LEFT };
}
```

```

// Convierte coordenadas de grilla (col, row) a posicion de mundo
public static Vector3 GridToWorld(Vector2Int g)
{
    return new Vector3(g.x * TILE_SIZE, -g.y * TILE_SIZE, 0f);
}

// Convierte una posicion del mundo al tile de grilla mas cercanos
public static Vector2Int WorldToGrid(Vector3 w)
{
    return new Vector2Int(
        Mathf.RoundToInt(w.x / TILE_SIZE),
        Mathf.RoundToInt(-w.y / TILE_SIZE)
    );
}

public static Vector2Int Opposite(Vector2Int dir) => -dir;

public static bool IsTunnelTile(Vector2Int g)
{
    return g.y == 14 && (g.x < 6 || g.x > 21);
}
}

```

Sobre el orden de prioridad de direcciones

El arreglo DIRS_CW (Clockwise: UP, RIGHT, DOWN, LEFT) define la prioridad de desempate cuando dos direcciones empatan en distancia al target. El arcade original usa el orden UP > LEFT > DOWN > RIGHT. Usaremos esta convención más adelante.

Paso 6.3

Crear el script MazeData

Define la disposición fija del laberinto. Crea Scripts/Core/MazeData.cs con el layout clásico de 28×31:

```

public static class MazeData
{
    /// <summary>
    /// Layout canónico del laberinto Pac-Man (28 columnas × 31 filas).
    /// # = muro, . = pellet, o = power pellet,
    /// espacio = transitable sin pellet, - = puerta de ghost house.
    /// </summary>
    public static readonly string[] LAYOUT = {

```

```

"#####",
"#.....#",
"#.###.###.###.###.###",
"#O###.###.###.###O#",
"#.###.###.###.###.###",
"#.....#",
"#.###.###.###.###.###",
"#.###.###.###.###.###",
"#.....#.....#.....#",
"##### ## #####",
"    #.### ## #####.#",
"    #.##          ##.#",
"    #.## ##—## ##.#",
"#####.## #    # ##.#####",
"    .    #    #    .    ",
"#####.## #    # ##.#####",
"    #.## ##### ##.#",
"    #.##          ##.#",
"    #.## ##### ##.#",
"#####.## ##### ##.#####",
"#.....#.....#.....#",
"#.###.###.###.###.###",
"#.###.###.###.###.###",
"#O..##.....#..O#",
"###.##.##.#####.##.##.###",
"###.##.##.#####.##.##.###",
"#.....#.....#.....#",
"#.#####.##.#####.###",
"#.#####.##.#####.###",
"#.....#.....#",
"#####",

```

```
};
```

```
}
```

Filas de túnel

Observa la fila 14 (" . # # . "): tiene los bordes laterales abiertos (con espacios al inicio y al final). Esos son los túneles laterales. Cuando Pac-Man o un fantasma sale por un lado, reaparece por el otro. Implementaremos esto en la sección 9.

Paso 6.4

Crear el generador del laberinto

El script MazeBuilder leerá MazeData.LAYOUT y generará tanto los GameObjects de los muros como una matriz lógica usada por la IA. Crea Scripts/World/MazeBuilder.cs:

```
using Unity.VisualScripting;
using UnityEngine;
using UnityEngine.Tilemaps;

public enum TileType { Wall, Empty, Pellet, PowerPellet, GhostDoor, GhostHouse };

public class MazeBuilder : MonoBehaviour
{
    [Header("Prefabs")]
    [SerializeField] private GameObject wallPrefab;
    [SerializeField] private GameObject pelletPrefab;
    [SerializeField] private GameObject powerPelletPrefab;
    [SerializeField] private GameObject ghostDoorPrefab;

    [Header("Referencias")]
    [SerializeField] private Transform wallsRoot;
    [SerializeField] private Transform pelletsRoot;

    // Matriz logica accesible para la IA
    public static TileType[,] Tiles { get; private set; }
    public static int TotalPellets { get; private set; }

    private void Awake()
    {
        BuildMaze();
    }

    private void BuildMaze()
    {
        Tiles = new TileType[GridConstants.COLS, GridConstants.ROWS];
        TotalPellets = 0;

        for (int row = 0; row < GridConstants.ROWS; row++)
        {
            string line = MazeData.LAYOUT[row];
            for (int col = 0; col < GridConstants.COLS; col++)
            {
                char c = col < line.Length ? line[col] : ' ';
                Vector3 pos = GridConstants.GridToWorld(new Vector2Int(col, row));
                switch (c)
                {
                    case '#':
                        Tiles[col, row] = TileType.Wall;
                        Instantiate(wallPrefab, pos, Quaternion.identity,
wallsRoot);
```

```

                break;
            case '.':
                Tiles[col, row] = TileType.Pellet;
                Instantiate(pelletPrefab, pos, Quaternion.identity,
pelletsRoot);

                TotalPellets++;
                break;
            case 'o':
                Tiles[col, row] = TileType.PowerPellet;
                Instantiate(powerPelletPrefab, pos, Quaternion.identity,
pelletsRoot);

                TotalPellets++;
                break;
            case '-':
                Tiles[col, row] = TileType.GhostDoor;
                if (ghostDoorPrefab != null)
                    Instantiate(ghostDoorPrefab, pos, Quaternion.identity,
wallsRoot);

                break;
            default:
                Tiles[col, row] = TileType.Empty;
                break;
        }
    }
}

public static bool IsWalkable(Vector2Int g, bool isGhost = false, bool
allowGhostDoor = false)
{
    if (g.x < 0 || g.x >= GridConstants.COLS || g.y < 0 || g.y >=
GridConstants.ROWS)
    {
        return g.y == 14; // solo el túnel permite salir del borde
    }
    TileType t = Tiles[g.x, g.y];
    if(t == TileType.Wall) return false;
    if(t == TileType.GhostDoor) return isGhost && allowGhostDoor;
    return true;
}
}

```

**Paso
6.5****Crear los prefabs visuales**

9. Crea un Quad: GameObject > 3D Object > Quad. Renómbralo a Wall.
10. Configura su Scale en (1, 1, 1).
11. Crea un material Mat_Wall con color azul Pac-Man (#2121DE) y asígnalo.
12. Arrastra el Wall a Assets/_Project/Prefabs para convertirlo en prefab. Elimínalo de la escena.
13. Repite el proceso para los demás prefabs:

Prefab	Forma	Escala	Color
Wall	Quad	(1, 1, 1)	#2121DE (azul)
Pellet	Sphere	(0.2, 0.2, 0.2)	#FFB897 (durazno)
PowerPellet	Sphere	(0.5, 0.5, 0.5)	#FFB897 (durazno)
GhostDoor	Quad	(1, 0.2, 1)	#FFB8FF (rosa claro)

Tip: prefab de pellet con trigger

Al Pellet y PowerPellet añádeles un componente CircleCollider2D con la opción Is Trigger marcada. Esto permitirá que Pac-Man los recolecte al colisionar. Configúralos también con el tag Pellet y PowerPellet respectivamente.

**Paso
6.6****Crear el GameObject MazeBuilder y probar**

14. En la jerarquía, crea un GameObject vacío llamado MazeBuilder.
15. Añádele el script MazeBuilder.
16. Crea dos hijos vacíos: Walls y Pellets (para organizar la jerarquía).
17. Asigna en el Inspector los prefabs (Wall, Pellet, PowerPellet, GhostDoor) y los transforms (Walls y Pellets).
18. Presiona Play. Deberías ver el laberinto clásico de Pac-Man generado dinámicamente.

7. Implementación de Pac-Man Jugable

Pac-Man se mueve de forma continua pero respetando la lógica de grilla. La técnica clave es el input buffering: cuando el jugador presiona una dirección, ésta se guarda y se aplica en cuanto el personaje esté alineado con un tile y la dirección no esté bloqueada por un muro.

Paso 7.1

Crear el GameObject Pacman

19. Crea una esfera: GameObject > 3D Object > Sphere.
20. Renómbrala a Pacman.
21. Configura su Transform:

```
Position: (13.5, -23, 0)
Rotation: (0, 0, 0)
Scale: (0.9, 0.9, 0.9)
```

22. Crea un material Mat_Pacman amarillo (#FFFF00) y asígnalo.
23. Añade un Rigidbody2D y configúralo con Body Type: Kinematic (no usaremos físicas, solo colisiones por código).
24. Añade un CircleCollider2D con Radius: 0.4.
25. Asigna el Tag Player.

Posición inicial

La posición (13.5, -23) corresponde al centro horizontal del laberinto entre las columnas 13 y 14, en la fila 23. Es la posición de spawn canónica de Pac-Man en el arcade.

Paso 7.2

Crear el script PacmanController

Crea Scripts/Player/PacmanController.cs con el siguiente contenido:

```
using UnityEngine;

public class PacmanController : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float speed = 5f;
```

```
[Header("Estado actual (debug)")]
[SerializeField] private Vector2Int currentDir = Vector2Int.zero;
[SerializeField] private Vector2Int queuedDir = Vector2Int.zero;

private Vector3 startPos;

public Vector2Int CurrentGrid => GridConstants.WorldToGrid(transform.position);
public Vector2Int CurrentDir => currentDir;

private void Awake()
{
    startPos = transform.position;
}

public void Respawn()
{
    transform.position = startPos;
    currentDir = Vector2Int.zero;
    queuedDir = Vector2Int.zero;
}

void Update()
{
    ReadInput();
    TryApplyQueuedDir();
    Move();
}

private void ReadInput()
{
    if (Input.GetKey(KeyCode.UpArrow) || Input.GetKey(KeyCode.W)) queuedDir =
GridConstants.UP;
    if (Input.GetKey(KeyCode.DownArrow) || Input.GetKey(KeyCode.S)) queuedDir =
GridConstants.DOWN;
    if (Input.GetKey(KeyCode.LeftArrow) || Input.GetKey(KeyCode.A)) queuedDir =
GridConstants.LEFT;
    if (Input.GetKey(KeyCode.RightArrow) || Input.GetKey(KeyCode.D)) queuedDir =
GridConstants.RIGHT;
}

private void TryApplyQueuedDir()
{
    if (queuedDir == Vector2Int.zero) return;

    Vector2Int g = CurrentGrid;
    Vector3 tileCenter = GridConstants.GridToWorld(g);
    float dist = Vector2.Distance(transform.position, tileCenter);
```

```

        // Solo cambiamos dirección cerca del centro del tile (tolerancia 0.1).
        // Si no hay dirección actual (inicio o detenido), omitir la restricción de
        distancia.
        if (currentDir != Vector2Int.zero && dist > 0.1f && queuedDir !=
GridConstants.Opposite(currentDir)) return;

        Vector2Int target = g + queuedDir;
        if (MazeBuilder.IsWalkable(target))
        {
            currentDir = queuedDir;
            // centrar para evitar drift acumulado
            if (queuedDir.x != 0) transform.position = new
Vector3(transform.position.x, tileCenter.y, 0);
            if (queuedDir.y != 0) transform.position = new Vector3(tileCenter.x,
transform.position.y, 0);
            queuedDir = Vector2Int.zero;
        }
    }

    private void Move()
    {
        if (currentDir == Vector2Int.zero) return;

        Vector3 delta = new Vector3(currentDir.x, -currentDir.y, 0) * (speed *
Time.deltaTime);
        Vector3 next = transform.position + delta;

        //Detener si el siguiente tile en la dirección actual es muro
        Vector2Int g = CurrentGrid;
        Vector2Int target = g + currentDir;
        Vector3 tileCenter = GridConstants.GridToWorld(g);
        float distToCenter = Vector2.Distance(transform.position, tileCenter);

        if (!MazeBuilder.IsWalkable(target) && distToCenter < 0.05f)
        {
            // Bloqueado contra muro: detener y centrar
            transform.position = tileCenter;
            currentDir = Vector2Int.zero;
            return;
        }

        transform.position = next;
    }
}

```

Sobre el signo del eje Y

Nota que en Move() multiplicamos currentDir.y por -1 al construir el delta. Esto es porque en nuestra grilla las filas crecen hacia abajo (Y positivo) pero en Unity el eje Y crece hacia arriba.

**Paso
7.3****Probar el movimiento**

26. Asigna PacmanController al GameObject Pacman.
27. Presiona Play.
28. Mueve a Pac-Man con flechas o WASD.
29. Verifica que: el movimiento es fluido, se detiene contra muros, puede invertir dirección instantáneamente, y solo puede girar en intersecciones.

8. Recolección de Pellets

Implementamos los scripts de los pellets y un sistema básico de scoring antes de pasar a los fantasmas (porque la activación de Inky y Clyde depende del contador de pellets).

Paso 8.1

Script del Pellet

Crea Scripts/World/Pellet.cs:

```
using UnityEngine;

public class Pellet : MonoBehaviour
{
    [SerializeField] protected int points = 10;
    [SerializeField] protected bool isPower = false;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.CompareTag("Player")) return;
        GameManager.Instance.OnPelletEaten(points, isPower);
        Destroy(gameObject);
    }
}
```

Para el PowerPellet, crea Scripts/World/PowerPellet.cs:

```
public class PowerPellet : Pellet
{
    private void Awake()
    {
        isPower = true;
        points = 50;
    }
}
```

Configuración rápida

En lugar de crear una clase derivada, puedes simplemente asignar el script Pellet a ambos prefabs y configurar manualmente isPower=true y points=50 en el prefab PowerPellet desde el Inspector.

**Paso
8.2****Script GameManager (esqueleto)**

Crea Scripts/Managers/GameManager.cs con la estructura básica. Lo ampliaremos en secciones siguientes:

```
using System.Collections;
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    [Header("Score")]
    [SerializeField] private int score = 0;
    [SerializeField] private int pelletsEaten = 0;

    public int PelletsEaten => pelletsEaten;
    public int Score => score;

    [Header("Vidas")]
    [SerializeField] private int lives = 3;
    public int Lives => lives;

    [Header("Referencias")]
    [SerializeField] private PacmanController pacman;
    [SerializeField] private GhostBase[] ghosts;

    public PacmanController Pacman => pacman;
    public GhostBase[] Ghosts => ghosts;

    private bool isDying = false;
    private int ghostEatenCombo = 0;

    private void Awake()
    {
        if (Instance == null) Instance = this;
        else Destroy(gameObject);
    }

    public void OnPelletEaten(int points, bool isPower)
    {
        score += points;
        pelletsEaten++;
        Debug.Log($"Score: {score} | Pellets: {pelletsEaten}/{MazeBuilder.TotalPellets}");
    }
}
```

```

        if (isPower)
        {
            foreach (var g in
FindObjectsByType<GhostBase>(FindObjectsSortMode.None)) g.EnterFrightened();
        }
        GhostHouseManager.Instance?.OnPelletEaten(pelletsEaten);
    }

    public void OnGhostEaten()
    {
        int points = 200 * (1 <= ghostEatenCombo); // 200, 400, 800, 1600
        score += points;
        ghostEatenCombo = Mathf.Min(ghostEatenCombo + 1, 3);
        Debug.Log($"¡Fantasma comido! +{points} Score: {score}");
    }

    public void ResetGhostCombo() => ghostEatenCombo = 0;

    public void OnPacmanDied()
    {
        if (isDying) return;
        isDying = true;
        lives--;
        Debug.Log($"Pac-Man murió. Vidas restantes: {lives}");
        StartCoroutine(RespawnSequence());
    }

    private IEnumerator RespawnSequence()
    {
        // Pausar a Pac-Man y fantasmas desactivando sus scripts.
        if (pacman != null) pacman.enabled = false;
        var allGhosts = FindObjectsByType<GhostBase>(FindObjectsSortMode.None);
        foreach (var g in allGhosts) g.enabled = false;

        yield return new WaitForSeconds(1.5f);

        if (lives <= 0)
        {
            Debug.Log("Game Over");
            isDying = false;
            yield break;
        }

        // Reiniciar posiciones.
        if (pacman != null) { pacman.Respawn(); pacman.enabled = true; }
        foreach (var g in allGhosts) { g.ResetToStart(); g.enabled = true; }
    }

```

```
// Reiniciar liberación de la ghost house.  
GhostHouseManager.Instance?.ResetReleaseState(pelletsEaten);  
  
isDying = false;  
}  
  
public Vector2Int GetBlinkyTile()  
{  
    foreach (var g in FindObjectsByType<GhostBase>(FindObjectsSortMode.None))  
        if (g is Blinky) return g.CurrentGrid;  
    return Vector2Int.zero;  
}  
}
```

Errores de compilación temporales

Este script hace referencia a GhostBase, Blinky y GhostHouseManager que aún no existen. Tendrás errores de compilación hasta que termines las siguientes secciones. Esto es normal; ignora los errores rojos por ahora.

Paso 8.3

Crear el GameObject GameManager

30. Crea un GameObject vacío llamado GameManager en la jerarquía.
31. Añádele el script GameManager.
32. Asigna Pacman al campo pacman del Inspector. Los Ghosts los asignaremos después.

9. Túneles Laterales (wrap-around)

La fila 14 del laberinto (la del medio) tiene ambos extremos abiertos. Cuando Pac-Man o un fantasma sale por el extremo izquierdo, debe reaparecer por el derecho y viceversa. Adicionalmente, los fantasmas se mueven más lentos en los tiles del túnel.

Paso 9.1

Actualizar PacmanController para el wrap-around

Modifica el método Move() de PacmanController añadiendo la lógica de wrap antes del return final:

```
// Al final de Move(), antes de "transform.position = next":
// Wrap-around en el túnel (fila 14).
const int TUNNEL_ROW = 14;
const float LEFT_EXIT = -1.5f;
const float RIGHT_EXIT = 28.5f;

if (Mathf.RoundToInt(-next.y) == TUNNEL_ROW)
{
    if (next.x < LEFT_EXIT) next.x = RIGHT_EXIT - 0.1f;
    if (next.x > RIGHT_EXIT) next.x = LEFT_EXIT + 0.1f;
}

transform.position = next;
```

Solución alternativa: trigger zones

También puedes crear dos GameObjects con BoxCollider2D (Is Trigger) en los extremos del túnel y manejar el wrap en OnTriggerEnter2D. Es más visual pero requiere más setup. Elige la que prefieras; la solución por código es más simple para esta guía.

Paso 9.2

Helper para detectar tiles de túnel

Añade un método estático a GridConstants:

```
public static bool IsTunnelTile(Vector2Int g)
{
    return g.y == 14 && (g.x < 6 || g.x > 21);
}
```

Lo usaremos en la sección de fantasmas para reducir su velocidad en estos tiles.

10. Clase Base de Fantasmas

Los cuatro fantasmas comparten la misma mecánica de movimiento y la misma máquina de estados. Solo difieren en el cálculo del target tile en modo Chase. Usaremos una clase base abstracta y cuatro clases hijas que sobrescriben el método `GetChaseTarget()`.

10.1 Diseño orientado a objetos

```
abstract class GhostBase {
    enum GhostState { Idle, InHouse, Leaving, Scatter, Chase,
        Frightened, Eaten }

    |
    |— Blinky : GhostBase → GetChaseTarget() = Pacman.tile
    |— Pinky  : GhostBase → GetChaseTarget() = Pacman.tile + 4 * Pacman.dir
    |— Inky   : GhostBase → GetChaseTarget() = 2*(Pacman.tile + 2*Pacman.dir) -
    Blinky.tile
    |— Clyde  : GhostBase → GetChaseTarget() = (dist > 8)? Pacman.tile : scatterCorner
}
```

Paso 10.2

Crear la enumeración de estados

Crea `Scripts/Ghosts/GhostState.cs`:

```
public enum GhostState
{
    InHouse,    // Dentro de la ghost house, esperando salir
    Leaving,    // Saliendo de la ghost house hacia el exit point
    Scatter,    // Hacia su esquina asignada
    Chase,      // Persiguiendo según su lógica personal
    Frightened, // Asustado (Pac-Man comió power pellet)
    Eaten       // Comido por Pac-Man, regresando a la ghost house
}
```

Paso 10.3

Crear la clase base GhostBase

Crea `Scripts/Ghosts/GhostBase.cs`. Es la clase más extensa de la guía; léela con cuidado:

```
using UnityEngine;
public abstract class GhostBase : MonoBehaviour
{
```

```

[Header("Velocidades (tiles/segundo)")]
[SerializeField] protected float normalSpeed = 5f;
[SerializeField] protected float frightenedSpeed = 2.5f;
[SerializeField] protected float tunnelSpeed = 3f;
[SerializeField] protected float eatenSpeed = 8f;

[Header("Esquina de Scatter")]
[SerializeField] protected Vector2Int scatterCorner;

[Header("Spawn dentro de la ghost house")]
[SerializeField] protected Vector2Int homeTile;

[Header("Estado actual (debug)")]
[SerializeField] protected GhostState state = GhostState.InHouse;
[SerializeField] protected Vector2Int currentDir;

public GhostState State => state;
public Vector2Int CurrentGrid => GridConstants.WorldToGrid(transform.position);
public Vector2Int CurrentDir => currentDir;

protected SpriteRenderer spriteRend;
protected Color originalColor;
protected Vector2Int targetTile;
protected Vector2Int lastChosenTile = new Vector2Int(-999, -999);

private Vector3 startPos;
private GhostState startState;

// Punto de salida de la ghost house (encima de la puerta).
protected static readonly Vector2Int GHOST_HOUSE_EXIT = new Vector2Int(13, 11);

protected virtual void Awake()
{
    spriteRend = GetComponentInChildren<SpriteRenderer>();
    if (spriteRend != null) originalColor = spriteRend.color;
    startPos = transform.position;
    startState = state;
}

public virtual void ResetToStart()
{
    transform.position = startPos;
    state = startState;
    currentDir = Vector2Int.zero;
    lastChosenTile = new Vector2Int(-999, -999);
    CancelInvoke(nameof(EndFrightened));
    CancelInvoke(nameof(LeaveHouse));
}

```

```

        if (spriteRend != null) spriteRend.color = originalColor;
    }

    protected virtual void Update()
    {
        UpdateTargetTile();
        Move();
    }

    /// <summary>
    /// Cada subclase implementa el cálculo de su target en modo Chase.
    /// </summary>
    protected abstract Vector2Int GetChaseTarget();

    /// <summary>
    /// Selecciona el target tile según el estado actual.
    /// </summary>
    protected virtual void UpdateTargetTile()
    {
        switch (state)
        {
            case GhostState.Chase: targetTile = GetChaseTarget(); break;
            case GhostState.Scatter: targetTile = scatterCorner; break;
            case GhostState.Frightened: targetTile = currentDir; break; // ignorado
en Move
            case GhostState.Eaten: targetTile = GHOST_HOUSE_EXIT; break;
            case GhostState.Leaving: targetTile = GHOST_HOUSE_EXIT; break;
            case GhostState.InHouse: /* dirección resuelta en ChooseNextDirection
*/ break;
        }
    }

    protected virtual void Move()
    {
        float speed = GetCurrentSpeed();
        Vector2Int g = CurrentGrid;
        Vector3 tileCenter = GridConstants.GridToWorld(g);
        float distToCenter = Vector2.Distance(transform.position, tileCenter);

        // Al llegar al centro de un tile (una sola vez por tile), elegimos
siguiente dirección.
        if (distToCenter < 0.05f && g != lastChosenTile)
        {
            transform.position = tileCenter;
            lastChosenTile = g;

            if (state == GhostState.Leaving && g == GHOST_HOUSE_EXIT)

```

```

        {
            bool scatter = ModeManager.Instance != null &&
ModeManager.Instance.IsScatter;
            state = scatter ? GhostState.Scatter : GhostState.Chase;
            currentDir = GridConstants.LEFT;
            return;
        }

        if (state == GhostState.Eaten && g == GHOST_HOUSE_EXIT)
        {
            state = GhostState.InHouse;
            currentDir = GridConstants.DOWN;
            if (spriteRend != null) spriteRend.color = originalColor;
            Invoke(nameof(LeaveHouse), 1f); // auto-sale tras llegar a casa
            return;
        }

        currentDir = ChooseNextDirection(g);
    }

    Vector3 delta = new Vector3(currentDir.x, -currentDir.y, 0) * (speed *
Time.deltaTime);
    Vector3 next = transform.position + delta;

    // Wrap del túnel.
    if (g.y == 14)
    {
        if (next.x < -1.5f) next.x = 28.4f;
        if (next.x > 28.5f) next.x = -1.4f;
    }

    transform.position = next;

    // Detección de colisión con Pac-Man.
    CheckPacmanCollision();
}

/// <summary>
/// Elige la dirección hacia el target tile minimizando distancia euclidiana,
/// con desempate por prioridad UP > LEFT > DOWN > RIGHT,
/// y prohibiendo la inversión de dirección.
/// </summary>
protected virtual Vector2Int ChooseNextDirection(Vector2Int g)
{
    if (state == GhostState.Frightened)
        return ChooseRandomDirection(g);
}

```

```

        // InHouse: rebote vertical – permite invertir dirección cuando está
        // bloqueado.
        if (state == GhostState.InHouse)
        {
            Vector2Int dir = currentDir != Vector2Int.zero ? currentDir :
GridConstants.DOWN;
            if (MazeBuilder.IsWalkable(g + dir, isGhost: true, allowGhostDoor:
false))
                return dir;
            return GridConstants.Opposite(dir);
        }

        Vector2Int[] priority = { GridConstants.UP, GridConstants.LEFT,
GridConstants.DOWN, GridConstants.RIGHT };

        Vector2Int best = currentDir;
        float bestDist = float.MaxValue;

        foreach (var dir in priority)
        {
            if (dir == GridConstants.Opposite(currentDir)) continue; // no invertir
            Vector2Int next = g + dir;
            bool ghostAllowed = (state == GhostState.Eaten || state ==
GhostState.Leaving);
            if (!MazeBuilder.IsWalkable(next, isGhost: true, allowGhostDoor:
ghostAllowed)) continue;

            float dist = Vector2.Distance(next, targetTile);
            if (dist < bestDist)
            {
                bestDist = dist;
                best = dir;
            }
        }
        return best;
    }

    protected Vector2Int ChooseRandomDirection(Vector2Int g)
    {
        System.Collections.Generic.List<Vector2Int> opts = new();
        foreach (var dir in GridConstants.DIRS_CW)
        {
            if (dir == GridConstants.Opposite(currentDir)) continue;
            if (!MazeBuilder.IsWalkable(g + dir, isGhost: true)) continue;
            opts.Add(dir);
        }
        if (opts.Count == 0) return GridConstants.Opposite(currentDir);
    }

```

```

        return opts[Random.Range(0, opts.Count)];
    }

    protected float GetCurrentSpeed()
    {
        if (state == GhostState.Frightened) return frightenedSpeed;
        if (state == GhostState.Eaten) return eatenSpeed;
        if (GridConstants.IsTunnelTile(CurrentGrid)) return tunnelSpeed;
        return normalSpeed;
    }

    // ===== Transiciones de estado =====

    public virtual void EnterScatter()
    {
        if (state == GhostState.InHouse || state == GhostState.Leaving ||
            state == GhostState.Eaten || state == GhostState.Frightened) return;
        if (state == GhostState.Chase || state == GhostState.Scatter)
            currentDir = GridConstants.Opposite(currentDir); // reversión en
transición
        state = GhostState.Scatter;
    }

    public virtual void EnterChase()
    {
        if (state == GhostState.InHouse || state == GhostState.Leaving ||
            state == GhostState.Eaten || state == GhostState.Frightened) return;
        if (state == GhostState.Chase || state == GhostState.Scatter)
            currentDir = GridConstants.Opposite(currentDir);
        state = GhostState.Chase;
    }

    public virtual void EnterFrightened()
    {
        if (state == GhostState.Eaten)
        {
            if (spriteRend != null) spriteRend.color = Color.green;
            return;
        }
        currentDir = GridConstants.Opposite(currentDir);
        state = GhostState.Frightened;
        if (spriteRend != null) spriteRend.color = Color.green;
        CancelInvoke(nameof(EndFrightened));
        Invoke(nameof(EndFrightened), 7f);
    }

    public virtual void EndFrightened()

```

```

{
    if (state != GhostState.Frightened) return;
    if (spriteRend != null) spriteRend.color = originalColor;
    bool scatter = ModeManager.Instance != null &&
ModeManager.Instance.IsScatter;
    state = scatter ? GhostState.Scatter : GhostState.Chase;
    GameManager.Instance?.ResetGhostCombo();
}

public virtual void GetEaten()
{
    state = GhostState.Eaten;
    if (spriteRend != null) spriteRend.color = new Color(1, 1, 1, 0.7f);
    // El target se actualiza automáticamente al GHOST_HOUSE_EXIT en
UpdateTargetTile.
}

public virtual void LeaveHouse()
{
    if (state != GhostState.InHouse) return;
    state = GhostState.Leaving;
}

// ===== Colisión con Pac-Man =====

protected void CheckPacmanCollision()
{
    var pacman = GameManager.Instance.Pacman;
    if (pacman == null) return;
    float dist = Vector2.Distance(transform.position,
pacman.transform.position);
    if (dist > 0.6f) return;

    if (state == GhostState.Frightened) { GameManager.Instance.OnGhostEaten();
GetEaten(); }
    else if (state != GhostState.Eaten) GameManager.Instance.OnPacmanDied();
}

// ===== Gizmos =====

private void OnDrawGizmosSelected()
{
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(GridConstants.GridToWorld(targetTile), 0.3f);
    Gizmos.DrawLine(transform.position, GridConstants.GridToWorld(targetTile));

    Gizmos.color = Color.red;

```



```
Gizmos.DrawWireCube(GridConstants.GridToWorld(scatterCorner), Vector3.one *  
0.8f);  
}  
}
```

El gizmo amarillo es tu mejor amigo

Cuando ejecutes el juego, selecciona un fantasma y verás una línea amarilla apuntando a su target tile actual. Esto es invaluable para entender por qué cada fantasma toma las decisiones que toma. Si tienes bugs, casi siempre los detectarás observando estos gizmos.

11. Implementación de los Cuatro Fantasmas

Ahora solo nos falta crear las cuatro subclases. Cada una sobrescribe el método `GetChaseTarget()` con la lógica canónica del fantasma.

Paso 11.1

Blinky (rojo)

Crea `Scripts/Ghosts/Blinky.cs`:

```
using UnityEngine;

public class Blinky : GhostBase
{
    [Header("Cruise Elroy")]
    [Tooltip("Si quedan menos pellets que este umbral, Blinky acelera.")]
    [SerializeField] private int elroyThreshold = 20;
    [SerializeField] private float elroySpeed = 6f;

    protected override Vector2Int GetChaseTarget()
    {
        var pacman = GameManager.Instance.Pacman;
        return pacman != null ? pacman.CurrentGrid : Vector2Int.zero;
    }

    protected override void Update()
    {
        base.Update();
        // Cruise Elroy: acelera cuando quedan pocos pellets.
        int remaining = MazeBuilder.TotalPellets -
            GameManager.Instance.PelletsEaten;
        if (remaining <= elroyThreshold && state == GhostState.Chase)
            normalSpeed = elroySpeed;
    }
}
```

Paso 11.2

Pinky (rosa)

Crea `Scripts/Ghosts/Pinky.cs`. Nota que reproducimos el bug de overflow del arcade:

```
using UnityEngine;

public class Pinky : GhostBase
{

```

```

protected override Vector2Int GetChaseTarget()
{
    var pacman = GameManager.Instance.Pacman;
    if (pacman == null) return Vector2Int.zero;

    Vector2Int p = pacman.CurrentGrid;
    Vector2Int d = pacman.CurrentDir;

    // 4 tiles delante de Pac-Man.
    Vector2Int target = p + d * 4;

    // Bug histórico: cuando Pac-Man mira UP, también se desplaza 4 a la
    // izquierda.
    if (d == GridConstants.UP)
        target += GridConstants.LEFT * 4;

    return target;
}
}

```

Reproducir bugs es parte de la fidelidad histórica

El "4 arriba 4 a la izquierda" no es un error de esta guía: es como funciona el arcade real desde 1980. Si lo "corriges" perderás fidelidad con el original. Esta es una decisión de diseño consciente, no un descuido.

Paso 11.3

Inky (cian)

Inky requiere conocer la posición de Blinky. Crea Scripts/Ghosts/Inky.cs:

```

using UnityEngine;

public class Inky : GhostBase
{
    protected override Vector2Int GetChaseTarget()
    {
        var pacman = GameManager.Instance.Pacman;
        if (pacman == null) return Vector2Int.zero;

        // Punto A: 2 tiles delante de Pac-Man.
        Vector2Int A = pacman.CurrentGrid + pacman.CurrentDir * 2;

        // (Reproducir bug similar al de Pinky cuando Pac-Man mira UP)
    }
}

```

```

        if (pacman.CurrentDir == GridConstants.UP)
            A += GridConstants.LEFT * 2;

        // Punto B: posición de Blinky.
        Vector2Int B = GameManager.Instance.GetBlinkyTile();

        // Target:  $A + (A - B) = 2A - B$ 
        return 2 * A - B;
    }
}

```

Paso 11.4

Clyde (naranja)

Crea Scripts/Ghosts/Clyde.cs:

```

using UnityEngine;

public class Clyde : GhostBase
{
    [Tooltip("Distancia (en tiles) bajo la cual Clyde huye en lugar de perseguir.")]
    [SerializeField] private float fearRadius = 8f;

    protected override Vector2Int GetChaseTarget()
    {
        var pacman = GameManager.Instance.Pacman;
        if (pacman == null) return scatterCorner;

        float dist = Vector2Int.Distance(CurrentGrid, pacman.CurrentGrid);
        if (dist > fearRadius) return pacman.CurrentGrid;
        return scatterCorner;
    }
}

```

Vector2Int.Distance no existe en versiones antiguas

Si tu versión de Unity no soporta `Vector2Int.Distance`, usa: `Vector2.Distance(CurrentGrid, pacman.CurrentGrid)`. El cast implícito de `Vector2Int` a `Vector2` funciona automáticamente.

12. Configuración Visual de los Fantasmas

Paso 12.1

Crear el prefab base

33. Crea una esfera: GameObject > 3D Object > Sphere. Renómbrala a Ghost.
34. Establece Scale en (0.9, 0.9, 0.9).
35. Añade un CircleCollider2D con Radius: 0.4 y la opción Is Trigger desmarcada.
36. Conviértela en prefab arrastrándola a Assets/_Project/Prefabs.
37. Crea 4 variantes del prefab (clic derecho > Create > Prefab Variant) y nómbralas: Blinky, Pinky, Inky, Clyde.

Paso 12.2

Asignar scripts, colores y posiciones

A cada variante asígnale su script correspondiente y los siguientes valores:

Fantasma	Color material	Home Tile (col, row)	Scatter Corner (col, row)
Blinky	#FF0000	(13, 11)	(25, 0)
Pinky	#FFB8FF	(13, 14)	(2, 0)
Inky	#00FFFF	(11, 14)	(27, 30)
Clyde	#FFB852	(15, 14)	(0, 30)

Las esquinas de Scatter caen sobre muros

Notarás que las coordenadas de Scatter (por ejemplo (25, 0) para Blinky) están en la esquina del laberinto donde hay muro. Esto es intencional: como ningún fantasma puede llegar al target, terminan dando vueltas alrededor de la esquina en un loop fijo, que es exactamente el comportamiento canónico del arcade.

Paso 12.3

Colocar los fantasmas en la escena

38. Arrastra las 4 variantes a la escena.

39. Configura sus Transform.position según el Home Tile (usa la conversión: $x = \text{col}$, $y = -\text{row}$, $z = 0$).
40. Por ejemplo, para Blinky (13, 11) $\rightarrow$ position = (13, -11, 0).
41. En el GameObject GameManager, asigna los 4 fantasmas al array Ghosts.

13. Mode Manager: Scatter/Chase Global

El ModeManager es un singleton que controla las transiciones globales Scatter↔Chase según los tiempos canónicos del arcade. Notifica a todos los fantasmas para que cambien de estado simultáneamente.

Paso 13.1

Crear el script ModeManager

Crea Scripts/Managers/ModeManager.cs:

```
using UnityEngine;

public class ModeManager : MonoBehaviour
{
    public static ModeManager Instance { get; private set; }

    [System.Serializable]
    public struct Phase
    {
        public bool isScatter;    // true=Scatter, false=Chase
        public float duration;    // segundos (-1 = permanente)
    }

    [Header("Fases canónicas del arcade (Nivel 1)")]
    [SerializeField]
    private Phase[] phases = new Phase[]
    {
        new Phase { isScatter = true,  duration = 7f },
        new Phase { isScatter = false, duration = 20f },
        new Phase { isScatter = true,  duration = 7f },
        new Phase { isScatter = false, duration = 20f },
        new Phase { isScatter = true,  duration = 5f },
        new Phase { isScatter = false, duration = 20f },
        new Phase { isScatter = true,  duration = 5f },
        new Phase { isScatter = false, duration = -1f } // Chase permanente
    };

    private int currentPhase = 0;
    private float phaseTimer = 0f;

    private void Awake()
    {
        if (Instance == null) Instance = this;
        else Destroy(gameObject);
    }
}
```

```

private void Start()
{
    ApplyCurrentPhase();
}

private void Update()
{
    if (phases[currentPhase].duration < 0f) return; // fase permanente

    phaseTimer += Time.deltaTime;
    if (phaseTimer >= phases[currentPhase].duration)
    {
        currentPhase++;
        phaseTimer = 0f;
        if (currentPhase >= phases.Length) currentPhase = phases.Length - 1;
        ApplyCurrentPhase();
    }
}

private void ApplyCurrentPhase()
{
    bool scatter = phases[currentPhase].isScatter;
    Debug.Log($"<color=yellow>Mode: {(scatter ? "SCATTER" : "CHASE")}</color>
(fase {currentPhase})");

    foreach (var g in GameManager.Instance.Ghosts)
    {
        if (scatter) g.EnterScatter();
        else g.EnterChase();
    }
}

public bool IsScatter => phases[currentPhase].isScatter;
}

```

**Paso
13.2****Añadir el ModeManager a la escena**

42. Crea un GameObject vacío llamado ModeManager.
43. Añádele el script ModeManager.
44. Los valores por defecto del Inspector ya corresponden al primer nivel del arcade.

Las fases en niveles superiores son más cortas

En el arcade original, a partir del nivel 5 las fases Scatter se acortan a 5s, 5s, 5s, 5s, 0s, ∞ — efectivamente eliminando los respiros Scatter. Como ejercicio puedes implementar un array de phases por nivel.

14. Ghost House Manager (orden de salida)

Los fantasmas no salen de la casa a la vez. El arcade usa contadores de pellets para escalonar su liberación. El GhostHouseManager se encarga de invocar LeaveHouse() en cada fantasma cuando se cumple su umbral.

Paso 14.1

Crear el script GhostHouseManager

Crea Scripts/Managers/GhostHouseManager.cs:

```
using UnityEngine;

public class GhostHouseManager : MonoBehaviour
{
    public static GhostHouseManager Instance { get; private set; }

    [Header("Umbrales canónicos (pellets comidos)")]
    [SerializeField] private int blinkyThreshold = 0;
    [SerializeField] private int pinkyThreshold = 0;
    [SerializeField] private int inkyThreshold = 30;
    [SerializeField] private int clydeThreshold = 60;

    [Header("Referencias")]
    [SerializeField] private GhostBase blinky;
    [SerializeField] private GhostBase pinky;
    [SerializeField] private GhostBase inky;
    [SerializeField] private GhostBase clyde;

    private bool blinkyReleased = false;
    private bool pinkyReleased = false;
    private bool inkyReleased = false;
    private bool clydeReleased = false;

    private void Awake()
    {
        if (Instance == null) Instance = this;
        else Destroy(gameObject);
    }

    private void Start()
    {
        // Blinky comienza ya fuera de la casa.
        if (blinky != null) { blinky.LeaveHouse(); blinkyReleased = true; }
        OnPelletEaten(0); // Pinky sale inmediatamente (umbral 0).
    }
}
```

```

    public void OnPelletEaten(int pelletsEaten)
    {
        if (!pinkyReleased && pelletsEaten >= pinkyThreshold) { pinky.LeaveHouse();
pinkyReleased = true; }
        if (!inkyReleased && pelletsEaten >= inkyThreshold) { inky.LeaveHouse();
inkyReleased = true; }
        if (!clydeReleased && pelletsEaten >= clydeThreshold) { clyde.LeaveHouse();
clydeReleased = true; }
    }

    public void ResetReleaseState(int currentPellets)
    {
        blinkyReleased = false;
        pinkyReleased = false;
        inkyReleased = false;
        clydeReleased = false;

        if (blinky != null) { blinky.LeaveHouse(); blinkyReleased = true; }
        OnPelletEaten(currentPellets);
    }
}

```

Movimiento dentro de la ghost house

Mientras un fantasma está InHouse oscila verticalmente en su tile (estética). Cuando se le llama LeaveHouse(), su estado cambia a Leaving y su target se fija a GHOST_HOUSE_EXIT (13, 11). El método ChooseNextDirection permite atravesar la GhostDoor solo cuando state==Leaving o state==Eaten. Esto está ya implementado en GhostBase. Si quieres animar el bobbing vertical durante InHouse, puedes sobrescribir Move() en cada fantasma o añadir una rama al Move base.

Paso 14.2

Configurar el GhostHouseManager

45. Crea un GameObject GhostHouseManager.
46. Añádele el script.
47. Arrastra los 4 fantasmas a los campos correspondientes.
48. Verifica los umbrales en el Inspector (0, 0, 30, 60).

15. Pruebas y Ajustes Finales

15.1 Checklist de funcionamiento

Ejecuta el proyecto y verifica los siguientes comportamientos uno por uno. Si alguno falla, revisa la sección correspondiente.

#	Comportamiento esperado	Sección
1	El laberinto se genera correctamente al iniciar.	6
2	Pac-Man se mueve fluidamente con flechas/WASD.	7
3	Pac-Man no atraviesa muros y se centra al chocar.	7
4	Pac-Man recolecta pellets (suma score en consola).	8
5	Pac-Man wrap-around en el túnel funciona.	9
6	Blinky sale al iniciar y persigue a Pac-Man directamente.	11.1
7	Pinky sale tras el primer pellet y emboscar 4 tiles adelante.	11.2
8	Inky sale tras 30 pellets y usa el cálculo $2A-B$.	11.3
9	Clyde sale tras 60 pellets y huye cuando se acerca.	11.4
10	Las transiciones Scatter↔Chase ocurren a los tiempos canónicos.	13
11	Power pellet activa Frightened: fantasmas azules, lentos, aleatorios.	10
12	Frightened termina tras 7 segundos.	10
13	Gizmos muestran target tiles al seleccionar un fantasma.	10

15.2 Problemas comunes y soluciones

Los fantasmas se quedan vibrando en una esquina

Causa probable: están atrapados oscilando entre dos tiles porque la prohibición de invertir dirección impide salir. Verifica que ChooseNextDirection nunca devuelva la dirección inversa salvo en transiciones explícitas. Si pasa solo cuando llegan al scatter corner, recuerda que ese comportamiento (loop perpetuo) es correcto.

Pac-Man se atasca al girar

Causa probable: la tolerancia en `TryApplyQueuedDir` es demasiado estricta. Aumenta la tolerancia de 0.1 a 0.15. También verifica que el `speed` sea un divisor entero de la distancia entre tiles para evitar overshoot.

Inky tiene comportamiento errático "raro"

Probablemente sea correcto. Inky es famoso por ser impredecible. Verifica con gizmos que el cálculo $2A-B$ produce un target lejos del laberinto a veces (eso es esperado). Si tienes dudas, fuerza Chase permanente y observa los gizmos: si la línea amarilla va a un punto coherente con el algoritmo (espejo de Blinky respecto a A), está correcto.

Los fantasmas atraviesan muros

Verifica que `MazeBuilder.IsWalkable` se está llamando con los parámetros correctos (`isGhost: true`, `allowGhostDoor: false` durante Chase y Scatter). El bug típico es marcar `allowGhostDoor=true` en todos los casos, lo que permite atravesar la puerta.

19. Recursos Complementarios

19.1 Referencias técnicas sobre la IA de Pac-Man

- Pittman, J. (2009). The Pac-Man Dossier. Disponible en gamedeveloper.com. La referencia técnica más completa sobre los algoritmos del arcade original.
- Birch, C. Understanding Pac-Man Ghost Behavior. Disponible en gameinternals.com (artículo clásico, ampliamente citado).
- Iwatani, T. Entrevistas históricas sobre el diseño del juego (múltiples disponibles online).

19.2 Documentación de Unity

- Tilemap: <https://docs.unity3d.com/Manual/Tilemap.html>
- Rigidbody2D Kinematic: <https://docs.unity3d.com/ScriptReference/Rigidbody2D.html>
- OnTriggerEnter2D: <https://docs.unity3d.com/ScriptReference/Collider2D.OnTriggerEnter2D.html>

19.3 Libros recomendados

- Millington, I., & Funge, J. (2009). Artificial Intelligence for Games (2nd ed.). Morgan Kaufmann.
- Buckland, M. (2005). Programming Game AI by Example. Wordware Publishing.
- Donovan, T. (2010). Replay: The History of Video Games. Yellow Ant. Capítulo sobre Pac-Man y la era arcade.

19.4 Videos recomendados

- Retro Game Mechanics Explained – Pac-Man Ghost AI Explained (YouTube). Análisis frame-by-frame del arcade.
- 8-Bit Guy – The History of Pac-Man (YouTube).
- Game Maker's Toolkit – Why Pac-Man's Ghosts Make the Game Great (YouTube).

Fin de la Guía de Laboratorio N° 10

INF761 — Desarrollo de Videojuegos

Universidad Autónoma Tomás Frías — Potosí, Bolivia